

IBM RPA Bot Generator

Iterative, validated generation of IBM RPA
WAL scripts

Build one subroutine at a time. Validate every command. Advance only when IBM RPA Studio is clean.

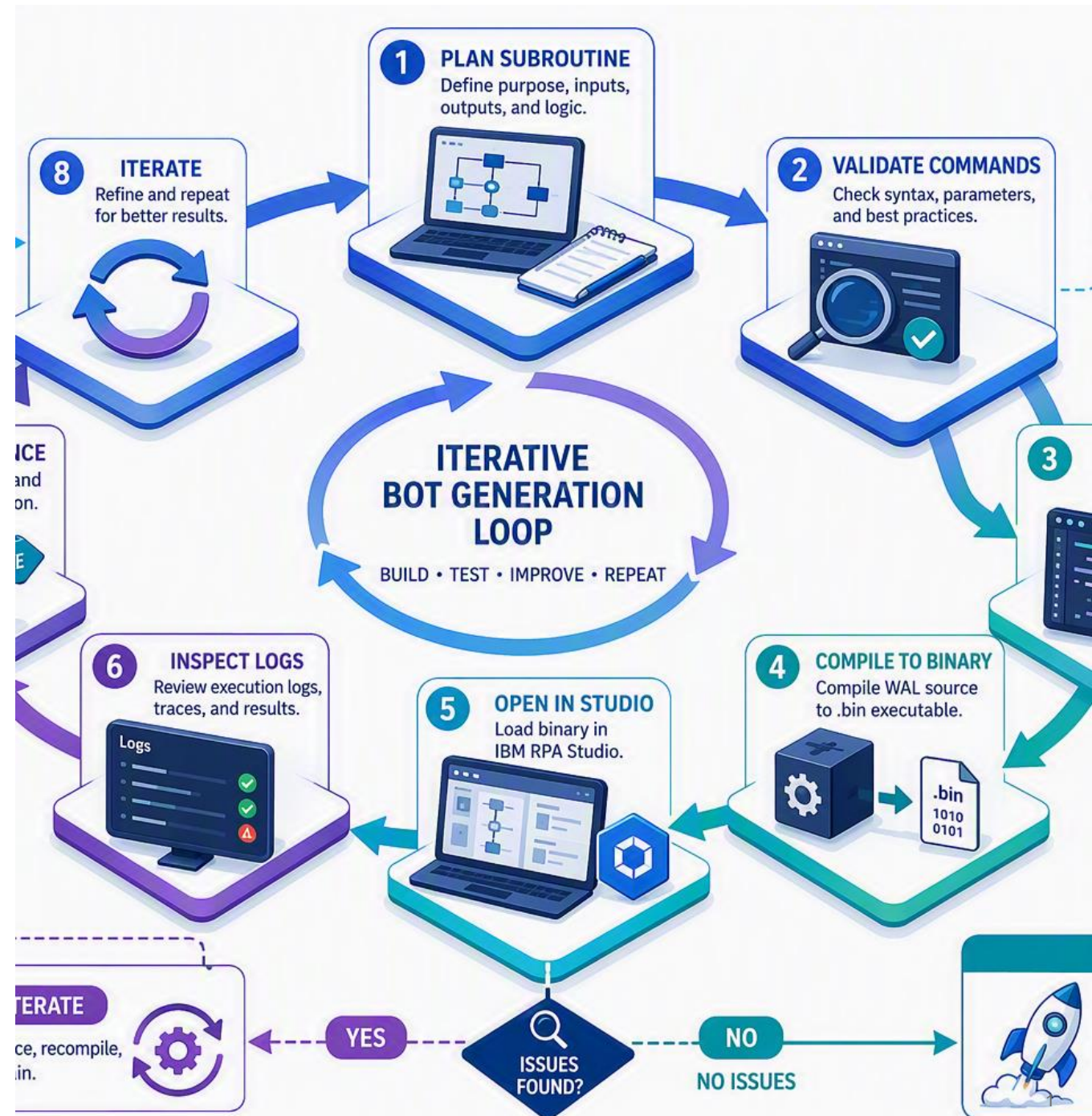
https://github.com/boontee/rpa_bot_generator

BOB MODE

WAL

STUDIO LOG LOOP

Technical overview



A governed workspace for reliable bot generation

The Bob mode combines source-of-truth commands, incremental development and Studio feedback.

1 Generate

Translate automation requirements into structured WAL source, using repeatable subroutine patterns.

2 Validate

Check every action, parameter and output variable against `commands_kb.md` before code is written.

3 Prove

Regenerate the binary, open it in Studio and use the Studio log as the acceptance gate.

Core operating principle

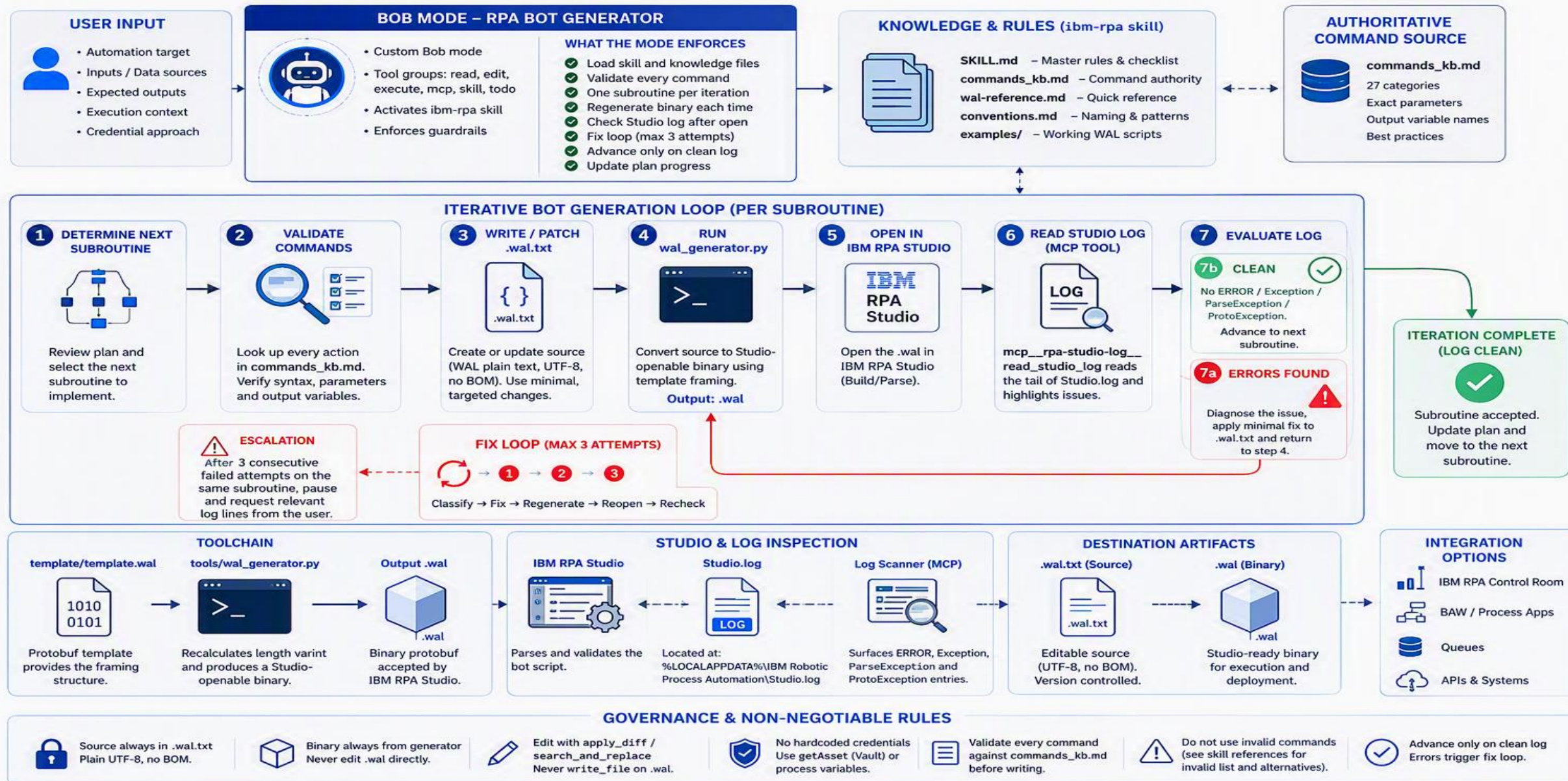
Exactly one subroutine per iteration

Errors trigger the minimal fix loop. A clean log unlocks the next subroutine.



IBM RPA BOT GENERATOR – ARCHITECTURAL DIAGRAM

Iterative, validated generation of IBM RPA WAL scripts



Three inputs start the guided workflow

The mode manages the generation mechanics after the automation intent is clear.

1

Select the mode

Choose RPA Bot Generator in the Bob mode picker.



2

Describe the bot

State the automation target, required inputs and expected outputs.



3

Follow the iterations

Review each generated subroutine and open the regenerated .wal in Studio.

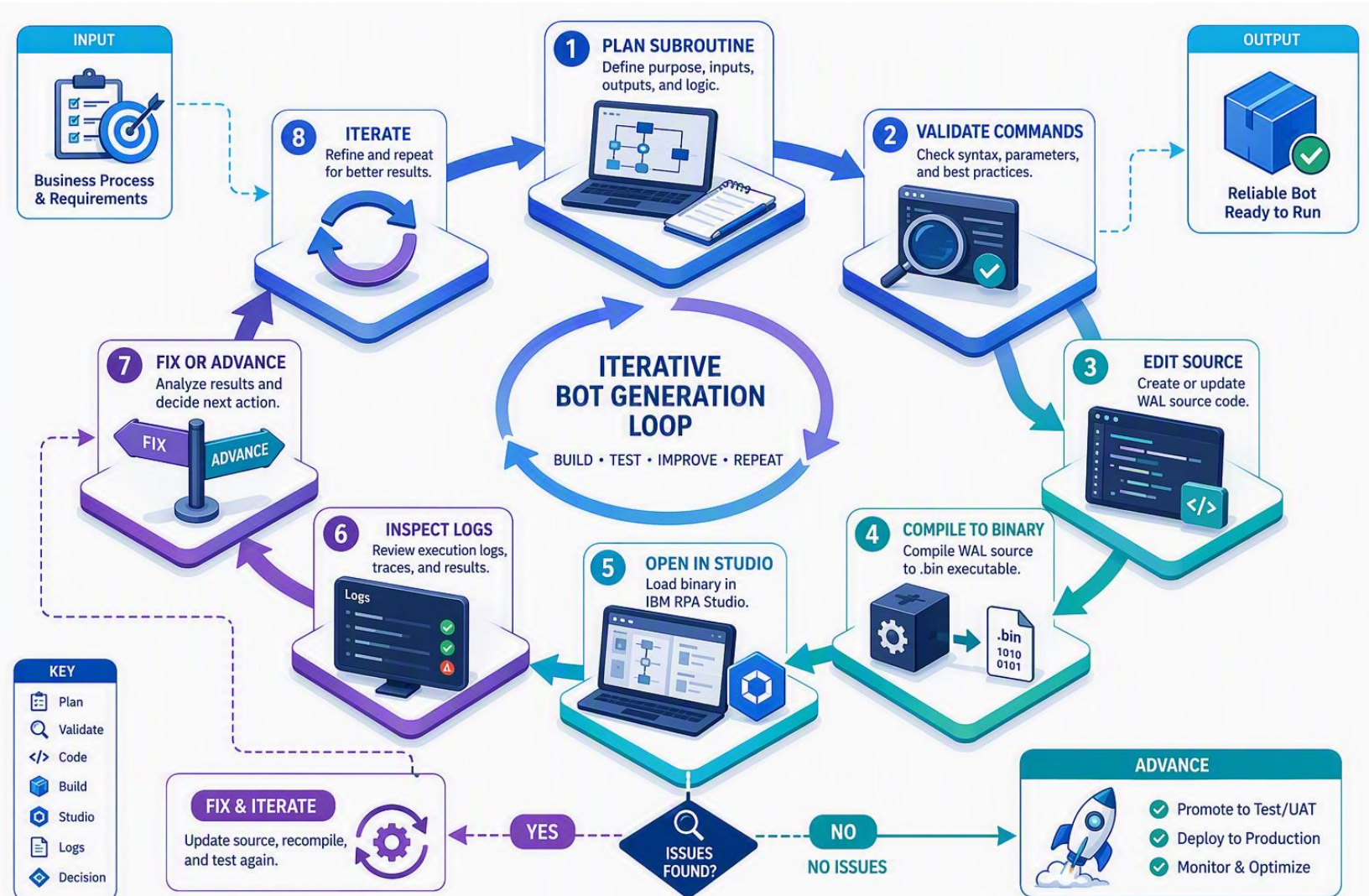
Minimum requirement set

Automation target • input sources • expected outputs • execution context • credential approach

RPA Bot Generator (Bob Mode Accelerator)

A closed loop turns WAL source into a Studio-clean bot

Each cycle produces evidence before additional logic is introduced.



Four phases control scope and quality

The process begins with a parseable skeleton and ends with dual-format handover.



Skeleton

All defVar declarations and stub beginSub/endSub blocks. Only entry and exit logs.



Subroutine

Fill one body. Validate commands, regenerate the binary and inspect the Studio log.



Fix loop

Classify the error, apply the smallest correction and rerun. Escalate after three failed attempts.

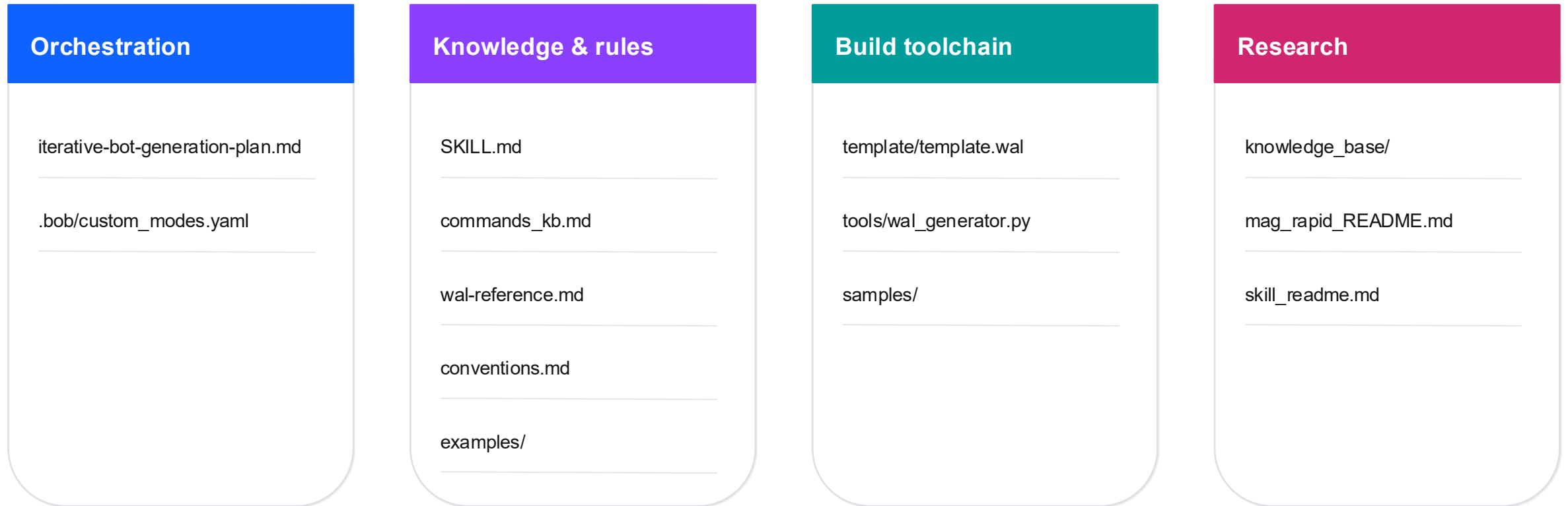


Final

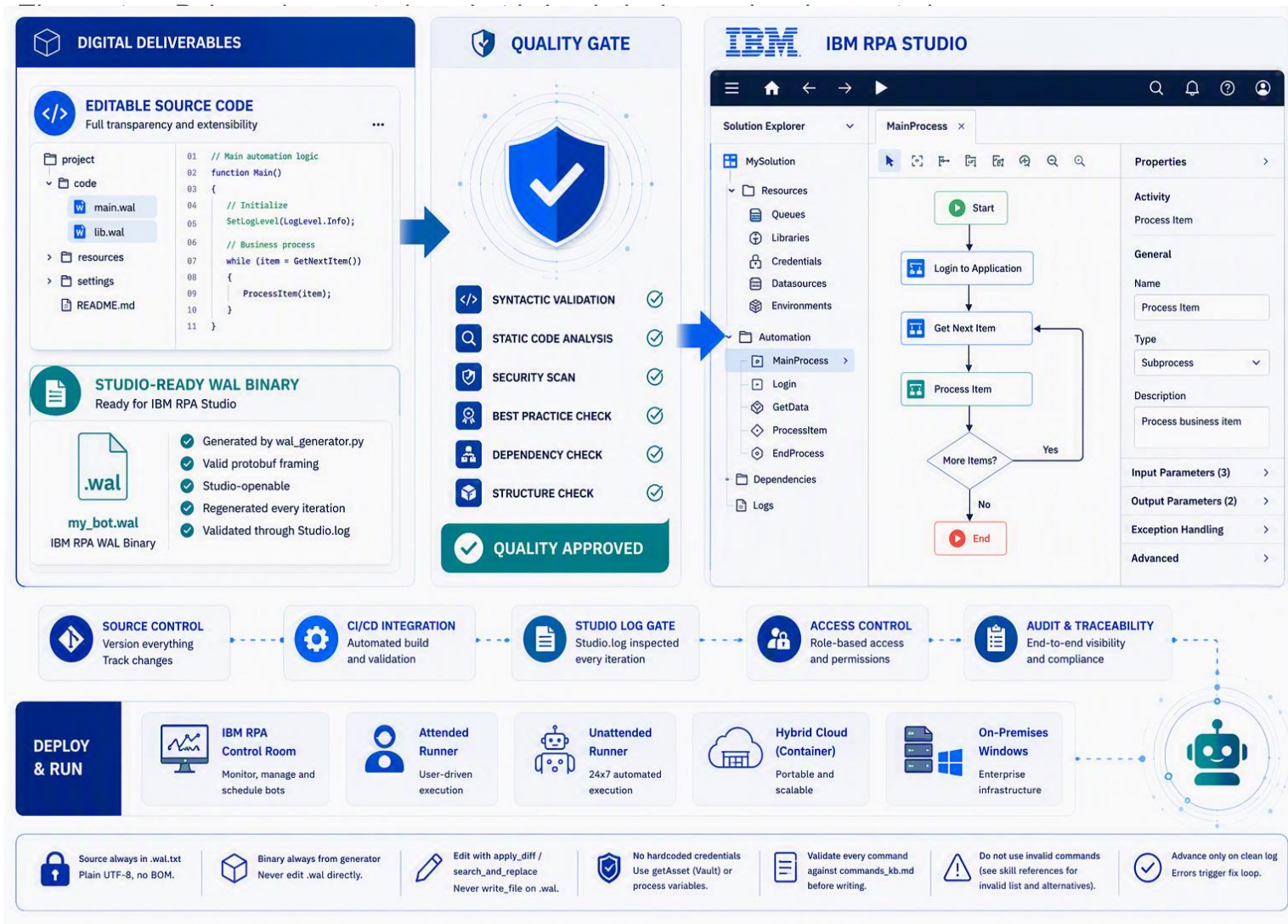
Run the complete checklist and deliver editable source plus Studio-ready binary.

The repository separates workflow, authority, tooling and outputs

Every file has a specific role in maintaining repeatability and traceability.



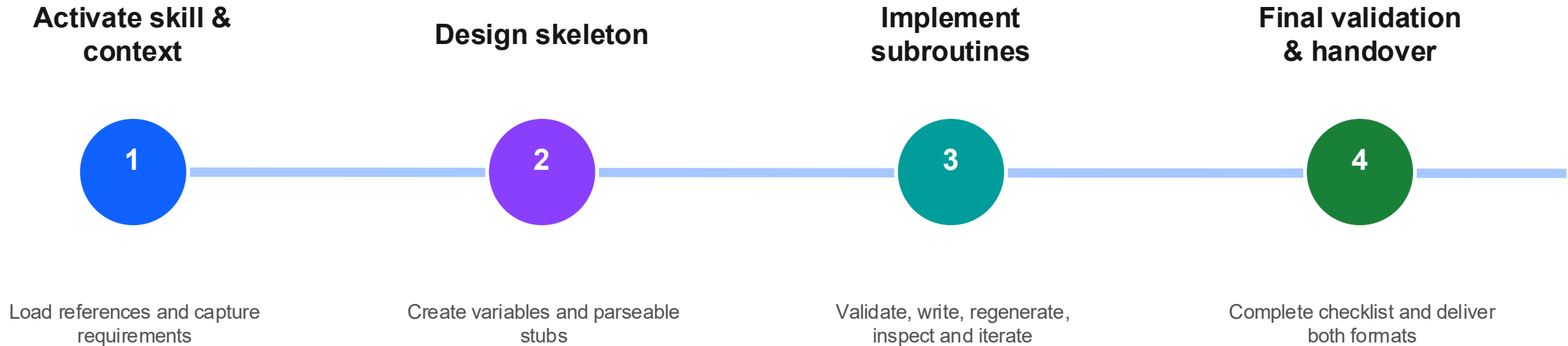
Guardrails keep generation aligned with the WAL source of truth



- ✓ Load the `ibm-rpa` skill at session start
- ✓ Read command and reference files before coding
- ✓ Resume from the tracked sub-task
- ✓ Implement exactly one subroutine per iteration
- ✓ Regenerate binary after every source change
- ✓ Inspect Studio log after every open
- ✓ Escalate after three failed fixes

Four sub-tasks define every bot generation session

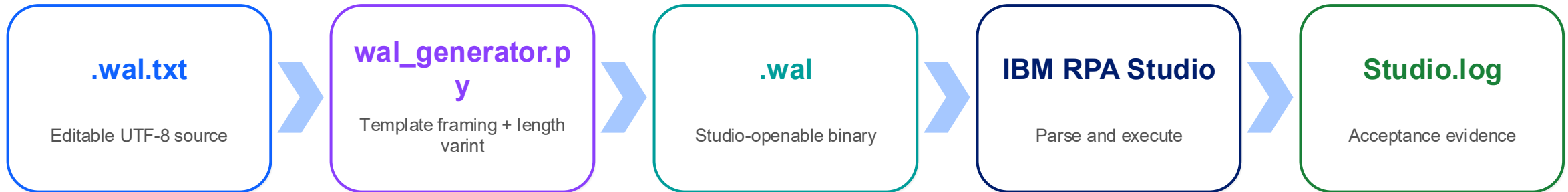
Status updates in the plan make the work resumable across sessions.



Resume point = last completed sub-task + current clean Studio state

Plain-text source is wrapped into a valid protobuf binary

The binary is always generated, never hand-edited.



Why the generator matters

A plain-text file renamed to **.wal** is not a valid protobuf package. The generator preserves the template framing and recalculates the length varint before Studio opens the file.

Clean log → advance

No command enters the script without an exact knowledge-base match

Action name, parameter names and output variable syntax are verified before generation.

Invalid	Use instead	Reason
throwException	throwError --message "..."	Studio-supported error command
getProcessVariable	getAsset or setVar defaults	Process variable command not supported
Quoted booleans	Unquoted booleans	Correct WAL literal syntax
setVar for arithmetic	evaluate --expression "..."	Use expression evaluation for calculations

If `commands_kb.md` does not list the command, stop and find a supported alternative.

Operational rules protect source integrity and prevent silent failures

Editable source, binary generation, credential handling and retry behavior are explicit.

Edit source only

Maintain plain UTF-8 .wal.txt without BOM. Use targeted patching rather than rewriting the binary.

Generate every time

Run wal_generator.py after every source modification. Never write a .wal directly.

Secure credentials

Use getAsset from the IBM RPA Vault or externally supplied process values. Never hardcode secrets.

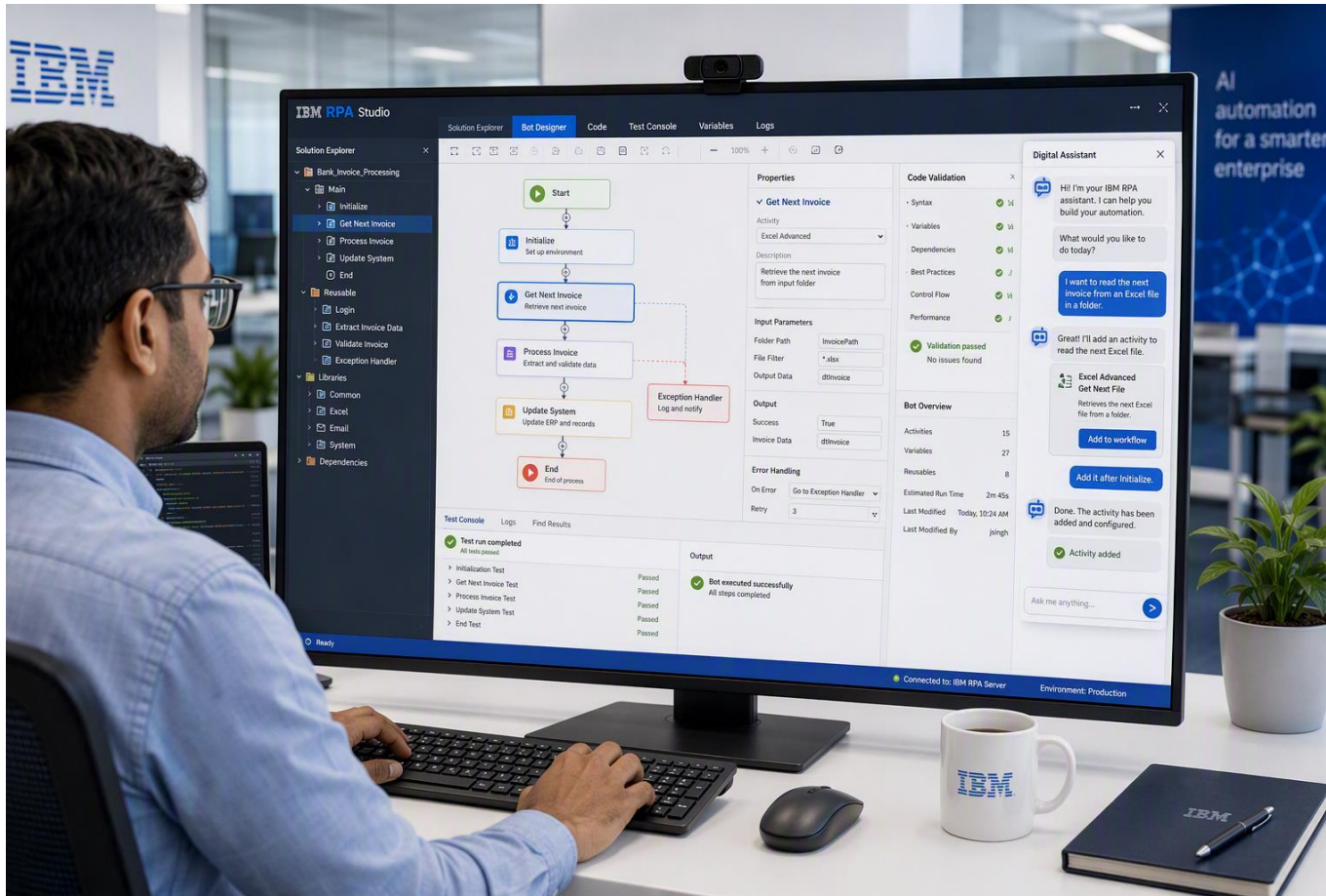
Error recovery protocol

- 1 Classify the Studio error
- 2 Apply the minimal fix
- 3 Regenerate and reopen
- 4 Re-read the log

After three consecutive failures on the same subroutine, pause and request the relevant log lines.

A complete bot ships with editable source and Studio-ready binary

The final gate confirms structure, syntax, generation and clean Studio feedback.



Final package

.wal.txt

Editable, reviewable source

.wal

Generated deployable binary

Assumptions

Inputs, constraints and dependencies

Limitations

Known gaps and escalation notes

Ready when the Studio log is clean

IBM RPA Bot Generator Thank You

Iterative, validated generation of IBM RPA
WAL scripts

**Build one subroutine at a time. Validate
every command. Advance only when IBM
RPA Studio is clean.**

https://github.com/boontee/rpa_bot_generator

BOB MODE

WAL

STUDIO LOG LOOP

Technical overview

